****How to develop user-defined custom Exception :**

As a developer we can develop user-defined checked OR user-defined unchecked exception.

If we want to develop checked exception then our user-defined class must extends from java.lang.Exception, on the other hand if we want to develop un-checked exception then our user-defined class must extends from
java.lang.RuntimeException.

In the user-defined exception class we should write No-argument constructor(in case if we don't want to pass any error message) and we should write parameterized constructor with String errorMessage as a parameter (in case if we want to pass any error message) with super keyword.

In order to throw the exception object explicitly we should use throw keyword as well as our user-defined class must be an instance of Throwable type.(Must be a Throwable Object)

```
package com.ravi.custom_exception;

import java.util.Scanner;

class InvalidAgeException extends Exception //Checked Exception
{
    //version compatibility [will see later in Serialization]
    private static final long serialVersionUID = 1L;

    public InvalidAgeException()
    {
        super();
    }

    public InvalidAgeException(String errorMessage)
    {
        super(errorMessage);
    }

}


public class CustomCheckedException {

    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        try{
            {
                IO.println("Enter your age :");
                int age = sc.nextInt();
                validAge(age);
            }
        catch(InvalidAgeException e)
            {
                IO.println(e);
            }
    }

    public static void validAge(int age) throws InvalidAgeException
    {
        if(age<18)
        {
            throw new InvalidAgeException("Age is Invalid");

        }
        else
        {
            IO.println("Your Age is :"+age);
            IO.println("You are eligible for Movie");
        }
    }

}
```

Custom Un-checked Exception :

```
package com.ravi.custom_exception;

class GreaterMarksException extends RuntimeException
{
    private static final long serialVersionUID = 1L;

    public GreaterMarksException()
    {
        super();
    }

    public GreaterMarksException(String errorMessage)
    {
        super(errorMessage);
    }
}

public class CustomUnchecked(Exception {

    public static void main(String[] args)
    {
        try
        {
            int marks = Integer.parseInt(IO.readln("Enter your marks :"));
            validateMarks(marks);
        }
        catch(GreaterMarksException e)
            {
                IO.println(e);
                IO.println("_____________________________________");
                IO.println("_____"+e.getMessage());
                IO.println("_____________________________________");
                e.printStackTrace();
            }


    public static void validateMarks(int marks)
    {
        if(marks>100)
        {
            throw new GreaterMarksException("Marks cannot be greater than 100");
        }
        else
        {
            IO.println("Your marks is :"+marks);
        }
    }
}
```

---

What is a Variable in Java ?

A variable is a name given for the memory location.

What is the drawback of an ordinary variable ?

An Ordinary variable can hold only one value at a time in sweden memory location.

Example :

int a = 10; //Valid
int a = 90.67; //Invalid

We want to hold multiple values in a contiguous memory location then we should use another concept called Array.

What is Array in Java :

An array is a container object that holds a fixed number of values of a single type. In ordered collection.

It can store multiple values in a single variable. In Java we can hold object type of data.

Example :-

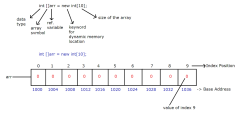
int [] a = {12,78,45,54,56}; //Homogeneous

Object [] obj = {1,"Ravi",null, 90.56}; //Object type

In Java array is an object so it is created as a part of HEAP memory which can hold primitive types as well as reference type. Since array is an object, it is created during runtime.

The length of an array is established when the array is created. After creation, its length is fixed.

An array Diagram for length 10 :







**Characteristics of an Array :**

1) Homogeneous Elements:

All elements in an array must be of the same data type only.

2) Fixed Size :

Once an array is created, its size cannot be changed (size is nothing but number of elements are available in the array).

3) Indexed Based:

We can store and access the elements of an array using index (arr[0]for the first element, arr[1] for the second, etc.)

a) How to find first and last index positive element
int []arr = {12,23,30,40,90,12}; //arr = 1000x
int first = arr[0]; //first = 1000
IO.println("Last Element :"+arr[arr.length-1]); //12

b) [Array array index starts from 0][and is taking 3 bytes of Memory]
1000 + 0 * 4 = 1000;//1000 is Base add, 0 is index & 4 ]
1000 + 1 * 4 = 1004
1000 + 2 * 4 = 1008;

4) Stored in Contiguous Memory:

Array elements are stored in adjacent memory locations, improving access speed (Cache Memory).

5) Stored in Heap Memory:

Arrays are objects in Java and are stored in the heap, even if declared inside a method.

6) Default Values:

If an array is declared but not initialized, Java assigns default values

7) Efficient Retrieval:

Direct access to elements using an index provides fast data retrieval (O(1) time complexity).

Drawback of an array :

1) Fixed in size
2) Insertion OR deletion at the begining or middle is O(n) time complexity
3) No predefined method support.

Working with single OR one Dimensional Array :

In Java we can create single dimensional array by using following ways :

Example 1 :

The most common way to create an array by using new keyword :

Example :
int[] arr = new int[5]; //Array of primitive
Student[] students = new Student[10]; //array of reference

Note : Here the array memory locations will be initialized with default
value.

Example 2 :

Another way to create an array object using new keyword with inline initialization
[The can initialize the array directly while creating it.]

Example :
int[] arr = new int[]{10, 20, 30, 40, 50};

Note : Here instead of default value user provides the value explicitly.

In the above example, expression is duplicate because the type of array we know from the left side of equal sign and since we are specifying the value of array hence we know the size also.

Instead of this we can use more convenient way which is known as
Anonymous array.

Example 3 :

int[] arr = {100, 200, 300};

Note : This is called anonymous array because we don't specify type and size.

Finally, We can use [] before or after the name, providing space is optional. The following 5 statements are same.

```
int[] num1;
int [] num2;//recommended
int []num3;
int num4[];
int num5[];
```

---

**What is Method chaining in Java ?**

* It is a technique through which we can call **multiple methods in a single statement.**
Example :
obj.m1().m2().m3().m4();

* In order to achieve method chaining that means to call another method, We need to depend upon the **last method return type.**

* We are calling multiple methods so, the final return type of the method will depend upon the **last method call**

Example of method chaining :

```
String str = "India";
char ch = str.concat(" is a great country").toUpperCase().charAt(0);
```

```
package com.ravi.method_chaining;

public class MethodChainingDemo1 {

    public static void main(String[] args)
    {
        String str = "India";
        char ch = str.concat(" is a great country").toUpperCase().charAt(0);
        IO.println(ch);
    }

}
package com.ravi.method_chaining;

public class MethodChainingDemo2 {

    public static void main(String[] args)
    {
        //Method Chaining
        char color = IO.readln("Enter color name :").toUpperCase().charAt(0);

        switch(color)
        {
        case 'R' -> IO.println("RED");
        case 'B' -> IO.println("Black");

        }

    }

}
```